easy-ssh: Local AI, Remote Execution

Develop locally with LLM, run on servers via SSH

Yusheng Zhao, Han Wang

HKUST(GZ)

2026-04-08

Outline

Easy-ssh

Appendix

The Problem

⚠️ Setting up LLM on a server

- API keys, network proxies, firewalls
- Different OS / package managers
- Every server needs its own config
- Debugging setup wastes hours

✨ But the server has

- GPU
- Larger RAM
- Faster CPU

Can we keep the LLM on the laptop and send only the *work* to the server?

The Solution: easy-ssh

A **single Bash script** - no installation needed on the server. Just SSH access.

Your Laptop

- Claude Code
- Codex
- Any LLM

SSH + rsync

push code, run command →

← pull results

Server (a800)

- GPU
- Big RAM
- Your data

edit locally → push → run remotely → pull results → repeat

Install - Let Your LLM Do It

You already have an LLM coding agent (Claude Code, Codex, etc.). **Use it.**

> *"Read the README at `github.com/exAClior/easy-ssh` and follow the instructions to install the CLI and the skill globally on my machine."*

The LLM will handle:

- Checking prerequisites (`ssh`, `rsync`, `bash`)
- Setting up passwordless SSH if needed
- Cloning the repo and adding cli to `PATH`
- Running `easy-ssh init` in your project

CLI Workflow — From Init to Results

① `easy-ssh init` — Interactive setup: prompts for host + remote dir, writes `.easy-ssh.conf`

↓

② `easy-ssh push` — rsync local code ⟶ server (additive — never deletes remote files)

↓

③ `easy-ssh run "cmd"` — Push + execute + **wait**. Streams stdout/stderr back to your terminal

↓

④ `easy-ssh submit "cmd"` — Push + launch via `nohup` + **return immediately**. Job survives disconnect

↓

⑤ `easy-ssh pull path/` — Fetch results from server ⟶ laptop

Also: `logs` (tail output) · `monitor` (live stream) · `status` (SSH + job state) · `clean` (prune remote)

What Gets Ignored During Push

`easy-ssh push` uses `rsync` under the hood. Three layers filter what gets synced:

① Default excludes (always skipped)

`.git/` and `.venv/` — hard-coded, no config needed

② .easy-ssh-ignore (your rules)

Same syntax as `.gitignore`. Example: `__pycache__/` `*.pyc` `data/` `*.h5` `results/` `node_modules/`

③ 500 MB safety guard

If effective sync size > 500 MB, push **refuses**. Fix: add ignore patterns or use `--force`.

Tip: Big datasets already on the server? Put them in `.easy-ssh-ignore`.
Cleanup: `push --clean` previews remote-only deletions; add `--force` to execute.

Live Demo — MQT YAQS on a800 (Python)

Real run: noisy analog simulation of an Ising chain using **MQT YAQS** (Tensor Jump Method).

```
easy-ssh init   # host='a800'  remote_dir='/home/yushengzhao/yaqs'
echo -e "__pycache__/\n*.pyc\ndata/\nresults/" > .easy-ssh-ignore
easy-ssh run "uv run python examples/analog_sim.py"
easy-ssh pull results/heatmap.png
```

✨ Key point

Zero setup on server — no API keys, no proxy, no conda env headaches.

Live Demo - Julia Quantum Simulation on a800

A real run: pulse simulation for three qubit gate (PulseODE.jl).

```
# Project already configured
cat .easy-ssh.conf
# host='a800'
# remote_dir='/home/yushengzhao/PulseODE.jl'

# One command: sync local code → run on server → see output
easy-ssh run "julia --project=. test/simulation.jl"

# Pull generated figures back to laptop
easy-ssh pull data/
```

Live Demo - What happened

Step 1: push rsync synced project files to `a800:/home/yushengzhao/PulseODE.jl/`

↓

Step 2: run Julia executed `simulation.jl` — three-qubit gate pulse simulation on server

↓

Step 3: pull Fetched simulation results + figures back to laptop

Result: Simulation data + plots in `data/` — ready for your paper.

Summary

What A single Bash script for local-AI + remote-execution

Why Keep LLM on laptop, send work to server — zero server setup

How `easy-ssh run "cmd"` syncs + runs + shows output

Install One `git clone` + add to PATH — done in 30 seconds

Server needs Just SSH access + rsync (already there)

Get started: `git clone git@github.com:exAClior/easy-ssh.git ~/.easy-ssh`

Summary

Questions?

Outline

Easy-ssh

Appendix

Commands at a Glance

- `easy-ssh init` — Interactive setup, writes `.easy-ssh.conf`
- `easy-ssh push` — Sync local files ⟶ server (additive, won't delete remote files)
- `easy-ssh run "cmd"` — Push + run + **wait** for output. Best for quick scripts.
- `easy-ssh submit "cmd"` — Push + launch in background + **return immediately**. Job survives disconnect.
- `easy-ssh logs` — Snapshot: last 50 lines of job output
- `easy-ssh monitor` — **Live-stream** job output; auto-stops when done
- `easy-ssh status` — Check SSH connection + job state
- `easy-ssh pull path` — Fetch files from server ⟶ laptop
- `easy-ssh clean` — Preview / remove remote-only files

Config — What NOT to Sync

Create `.easy-ssh-ignore` to skip big files (same syntax as `.gitignore`):

```
# .easy-ssh-ignore
__pycache__/
*.pyc
.venv/
data/
*.h5
node_modules/
.git/
```

Why this matters:

- Without it, `push` uploads **everything** — including GBs of data
- The server probably already has the data at a known path
- Keeps sync fast (seconds, not minutes)

Config — What NOT to Sync

Safety: easy-ssh refuses to sync if your directory is > 500 MB. Add ignore patterns or use `--force` to override.

Bonus — Claude Code Integration

Give Claude Code the ability to use easy-ssh on your behalf:

```
npx skills add exAClior/easy-ssh --skill easy-ssh
```

Now you can say things in natural language:

> *"Run train.py on the server"*
> *"Submit a training job with 50 epochs"*
> *"Pull the results from the server"*
> *"Check if my job is still running"*

Claude Code handles `push`, `run`, `submit`, `pull`, `monitor`, `status`, and `logs` automatically.